# High-Level Design Document

**VueXR Studio — AI-Powered XR Content Creation Desktop Application**

**Platforms:** Windows (x64), macOS (Apple Silicon preferred)
**Frontend Framework:** Unity Editor 6.2.x (Standalone App)
**Backend/Bridge:** Python plugin for AI model integration

## 1. Project Overview

VueXR Studio is a **local AI-driven XR asset creation tool** designed to generate 3D models using open-source AI frameworks. The application allows users to type prompts, generate 3D content locally, view/edit/inspect it in a 3D viewport, and publish or export assets to VueXR or local storage.

This tool is a core component of the broader **VueXR ecosystem**, enabling a "Create → Publish → Experience" pipeline without cloud inference cost.

## 2. Core Objectives

| Objective | Description |
|---|---|
| AI-Generated XR Assets | Convert text prompts into 3D meshes using local generative AI models |
| Cross-Device Unity Experience | Modern, responsive UI built using Unity runtime UI toolkit |
| Offline-First Architecture | All inference runs locally, ensuring privacy and zero cloud cost |
| Publishing-to-Platform | One-click export and publish to VueXR platform |
| Cross-OS Consistency | Identical UX and output flow for Windows and macOS environments |

## 3. Functional Requirements

### 3.1 User Interface

**Window Structure:**

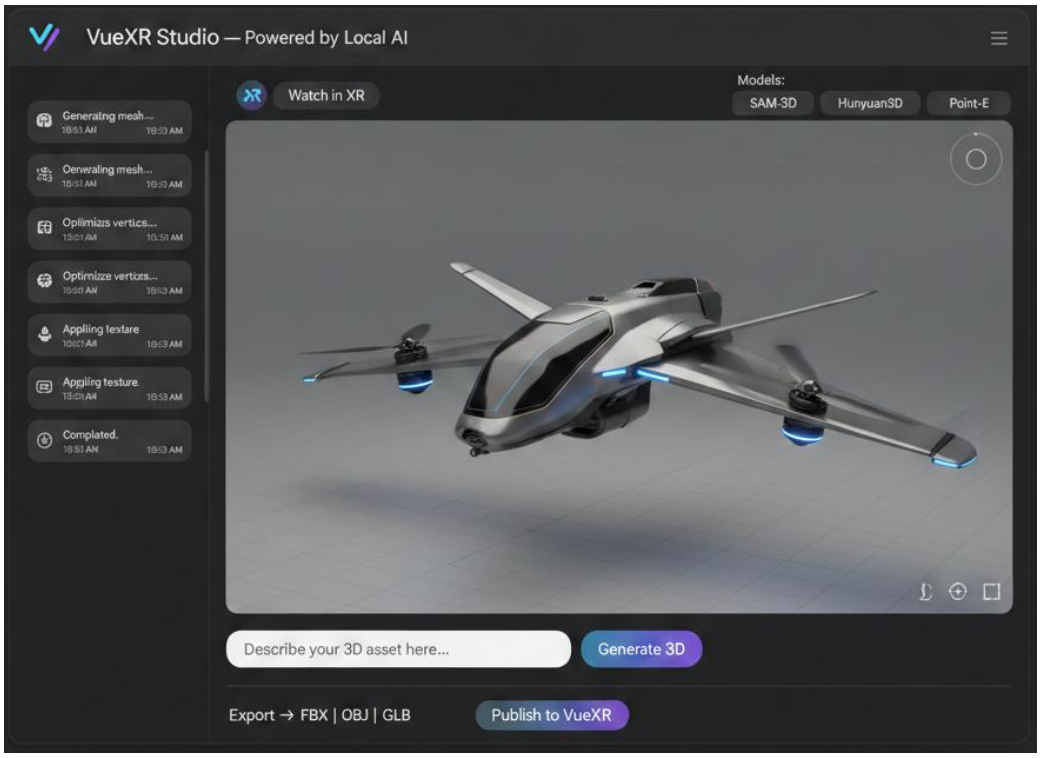
Left Panel: Chat History

Main Area:

- Top Right: 3D Viewer (largest section)

- Top Left: Model/Settings Dropdown + Watch in XR

- Bottom Panel: Prompt Text Box + Submit Button




**UI Components:**

| UI Element | Function |
|---|---|
| **Chat History Panel (Left)** | Scrollable list of all previous prompt interactions |
| **Model Selection Dropdown (Top Right)** | Choose active model: SAM-3D, Hunyuan3D, Point-E (extendable) |
| **Prompt Text Box (Bottom)** | Accepts user input (press Enter or "Generate") |
| **3D Viewer** | Displays generated assets with orbit, zoom, pan controls |
| **“Watch in XR” Button (Top Left)** | Popup for publishing workflow |

---

## 3.2 Publishing Workflow (Watch in XR Popup)

On click → Open modal:

Fields:

- Title *(Required)*
- Description
- Tags (Comma separated)
- Metadata auto-fill: poly count, texture resolution, file size

- Export format selection: **FBX, OBJ, GLB**

Actions:

- **Preview**
- **Publish to VueXR**
- **Save locally**
- **Cancel**

On publish: Upload via secure REST API / Token authentication to VueXR cloud publishing endpoint.

---

## 3.3 AI Model Processing Flow

Unity (Prompt Text)

↓

Python Bridge (API or Socket Interface)

↓

AI Model Engine (Locally Installed Model)

↓

Generated Asset Output (Mesh/Texture/Metadata)

↓

Unity Runtime → Display + Save + Export

**Requirements:**

- Use Python virtual environment packaged with app
- Support multiple model backends (modular architecture)
- All inference runs locally (offline mode first)
- GPU acceleration mandatory (CUDA/Metal)

---

## 3.4 Persistence Requirements

| Type | Location | Notes |
|---|---|---|
| Prompt History | Local DB (SQLite recommended) | Indexed by timestamp and model name |
| User Account Sync | VueXR Cloud Sync (optional) | When logged in, sync history across devices |
| Generated Models | Local storage (~/Documents/VueXR/Exports) | Respect license terms |

---

### 3.5 Export Requirements

Supported formats:

- **GLB (preferred)**
- **FBX**
- **OBJ**

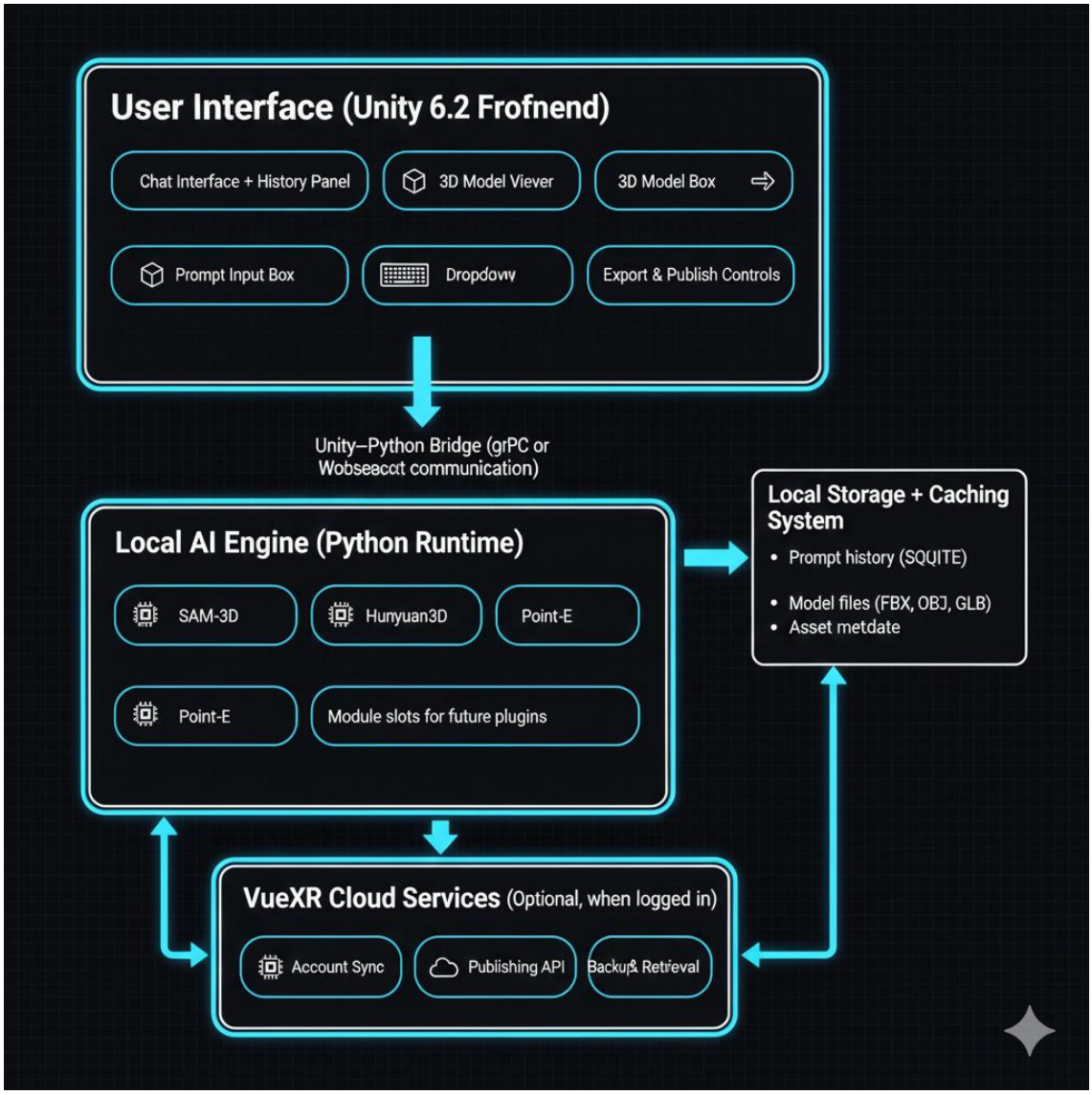
Mesh compression supported: **Draco / Mesh-opt** (optional).
Textures: **PNG/WebP**, optional **PBR maps**.

---

## 4. System Architecture

### 4.1 Architectural Paradigm

- **Modular, isolated responsibilities**
- **Unity handles UI + rendering**
- **Python handles AI logic + inference**
- **Shared messaging layer for communication**




---

## 5. Non-Functional Requirements

| Category | Requirement |
|---|---|
| Performance | Generate preview in < 30 seconds on GPU-enabled laptop |
| Offline Mode | Fully functional without internet |
| Scalability | Support plugin-based AI model additions |
| Portability | Single codebase for Windows + macOS |

---

## 6. Legal & Compliance

- Only open-source models (Apache/MIT-compatible) permitted.
- All license terms must appear during onboarding.
- Models installed locally under user control to avoid IP exposure.

## 7. Risks & Mitigation

| Risk | Mitigation |
|---|---|
| Model size impacts boot time | Offer optional model download after install |
| GPU compatibility issues | Built-in diagnostics tool + fallback inference mode |
| User confusion on exports | Guided UX with presets and validation |

## 8. Deliverables

| Phase | Output |
|---|---|
| Sprint 1 | UI shell + Python bridge |
| Sprint 2 | Model inference + asset preview |
| Sprint 3 | Export pipeline + publishing |
| Sprint 4 | Sync, polish, QA, release candidate |

## 9. Success Criteria

- < 5 clicks from prompt → usable 3D model
- 100% offline inference
- Publish-to-VueXR flow < 30 sec end-to-end
- Exported asset should be usable in Unity, Blender, Unreal